

9. Bölüm

Temsilciler, Lambda İfadeleri ve Olaylar

9.1 Temsilciler

Temsilci (**delegate**), yöntemleri dolaylı olarak çağırmanın bir yoludur. Saf nesne yönelimli programlama dillerinde (Java ve C#), yarattığı karmaşadan ötürü gösterici (**pointer**) kullanımı tercih edilmez. Bu sebeple C++ dilindeki fonksiyon göstericisi (**function pointer**) yerine, ondan daha yetenekli olan temsilciler kullanılır.

Temsilciler, C#'taki olay tabanlı programlamanın (**event-driven programming**) ve fonksiyonel programlama (**functional programming**) yeteneklerinin temelidir.

Temsilciler, yöntemlerin (**method**) bellek adreslerini tutan tip güvenli (**type-safe**) yapılardır. Temsilciye bağlanacak yöntemin imzası (**signature**) temsilci tanımında verilir. Yöntem imzası kavramı 8.4.9 başlığında verilmiştir. Bir temsilciye ilk yöntem atama (=) işleciyle bağlanır. Geri kalan yöntemler += işleci ile bağlanır. **Bir temsilciyi çağırdığınızda, ona bağlı olan tüm yöntemler eklendiği sırayla çalıştırılır FIFO (first in first out)!**

```
// 1. Temsilci Tanımı
public delegate void Yap(string pİş); //(İmza: Geridönüş tipi: void, parametre: string olacak!)

//Temsilciler Sınıflar Olmadan Çalışmaz!. Bu nedenle "Main" Fonksiyonu içeren bir sınıf mutlaka
→ olmalı!
public class Program
{
    public static void Main()
    {
        YöntemliSınıf nesne = new YöntemliSınıf();
        // 2. Temsilci Örneği Oluşturma ve İlk Yöntemi Atama
        Yap pİşyap = nesne.MusluğuAç;
        // 3. Çoklu Gönderim (Multicast): Yeni yöntemler ekleme
        pİşyap += nesne.MusluğuKapat;
        pİşyap += YöntemliSınıf.BulaşıkYık; // Statik yöntem ekleme
        // 4. Temsilciyi Çağırma (Invocation)
        Console.WriteLine("--- Temsilci Çalıştırılıyor ---");
        pİşyap("İlhan Özkan");
        // 5. Yöntem Çıkarma (İstenirse bir yöntem listeden çıkarılabilir)
        pİşyap -= nesne.MusluğuAç;
    }
}

/* Program Çıktısı:
--- Temsilci Çalıştırılıyor ---
İlhan Özkan Musluğu Açtı.
İlhan Özkan Musluğu Kapattı.
İlhan Özkan Bulaşık Yıkadı.
*/

public class YöntemliSınıf
{
    // Örnek (Instance) Yöntemi
    public void MusluğuAç(string pİsim)
    {
        Console.WriteLine($"{pİsim} Musluğu Açtı.");
    }

    public void MusluğuKapat(string pİsim)
    {
        Console.WriteLine($"{pİsim} Musluğu Kapattı.");
    }
}
```

```

    }

    // Statik Yöntem
    public static void BulaşıkYıka(string pİsim)
    {
        Console.WriteLine($"{pİsim} Bulaşık Yıkadı.");
    }
}

```

9.2 Lambda İfadeleri

§9.2.1 Lambda İfadesi Nedir?

Bir **lambda ifadesi** (**lambda expression**), basit fonksiyon nesneleri oluşturmak için özlü bir yol sağlar. **lambda** adı, *Alonzo Church* tarafından 1930'larda mantık ve hesaplanabilme hakkındaki soruları araştırmak için icat edilen matematiksel bir form olan lambda hesabından gelir. lambda hesabı, fonksiyonel bir programlama dili olan LISP'in temelini oluşturmuştur.

C# dilindeki lambda ifadeleri, kodunuzu daha kısa, daha okunabilir ve daha fonksiyonel bir hale getirir. lambda ifadeleri, aslında **isimsiz yöntemler** (**anonymous method**) yazmanın en kestirme yoludur. lambda ifadelerinde girdi parametrelerinin veri tipini belirtmenize gerek yoktur. Bu da kullanımı daha esnek hale getirir.

§9.2.2 Lambda Nasıl Tanımlanır?

lambda ifadesi iki bölüme ayrılır, sol taraf girdidir, sonrasında **lambda işleci** (**lambda operator**) (**=>**) bulunur. Sağ taraf ise ifadedir (**expression**).

Talimatlar (**statement**), yapılması gerekeni kısa ve net olarak belirten mantıksal cümlelerdir (**logical sequences**). Talimatlar anahtar kelimeler içerirler ve noktalı virgül karakteri ile biter. **İfadelerin** (**expression**) ise yalnızca sabit, değişken ve işleç (**operator**) içeren talimatları olduğu daha önce anlatılmıştı.

Lambda ifadeleri, temsilcilere veya ileride göreceğimiz LINQ sorgularına atanan anonim fonksiyonlardır. Söz diziminde kullanılan lambda işleci, "şuna gider" veya "için şunu yap" anlamı taşır. Genel formül aşağıda verilmiştir;

$$(\text{parametreler}) \Rightarrow \{\text{yapılacak işlemler}\}$$

Lambda ifadelerinin söz dizimi iki türlüdür:

1. **İfade lambdaları** (**expression lambdas**): Sadece tek bir işlem yapan ve sonucu otomatik dönen kısa yapılardır.
`(x) => x * x; // x parametresini alır ve karesini döner.`
2. **Talimat lambdaları** (**statement lambdas**): Süslü parantez { } kullanarak birden fazla işlem satırı içeren yapılardır.
`(x, y) => {
 int toplam = x + y;
 Console.WriteLine(toplam);
 return toplam;
};`

§9.2.3 Temsilcilerle Lambda Kullanımı

Daha önce örneğini verdiğimiz **Yap** temsilcisini hatırlayalım. Onu lambda ile aşağıdaki gibi daha kısa yazabiliriz. Örnekte başka temsilci örneği de bulunmaktadır;

```

public delegate void Yap(string pİş); // Temsilci tanımı

public delegate int intGirdiAlanveİntDönenBirTemsilci(int input);

class Program {
    static void Main() {
        // Eski yöntem (Anonim yöntem - C#2.0)
        Yap islem1 = delegate(string mesaj) { Console.WriteLine(mesaj); };
        // Yeni yöntem (Lambda ifadesi - C#3.0+)
        Yap islem2 = (mesaj) => Console.WriteLine(mesaj);
        islem2("İlhan Özkan işe başladı.");

        intGirdiAlanveİntDönenBirTemsilci ikiİleÇarp = x => x * 2;
    }
}

```

```

intGirdiAlanveIntDönerBirTemsilci karesiniAl = x => x * x;
intGirdiAlanveIntDönerBirTemsilci küpünüAl = x => x * x * x;
int sayi = 10;
Console.WriteLine("10*2 = " + ikiİleÇarp(10)); // 20
Console.WriteLine("sayi^2 = " + karesiniAl(sayi)); //100
Console.WriteLine("sayi^3 = " + küpünüAl(sayi)); //1000
}
}

```

Anonim yöntemler (**anonymous method**) bir kod bloğunu temsilci (**delegate**) parametresi olarak geçirmek için bir teknik sağlar. Bunlar bir gövdeye sahip ancak adı olmayan yöntemlerdir ve lambda ifadeleri ile tanımlanırlar.

```
delegate int IntOp(int lhs, int rhs);
```

```
class Program
```

```

{
    static void Main(string[] args)
    {
        // C#2.0 Tanımlaması
        IntOp add = delegate(int lhs, int rhs)
        {
            return lhs + rhs;
        };
        // C#3.0 Uzun Tanımlama
        IntOp mul = (lhs, rhs) =>
        {
            return lhs * rhs;
        };
        // C#3.0+ Kısa Tanımlama
        IntOp sub = (lhs, rhs) => lhs - rhs;

        // Her bir anonim yöntemi çağıralım:
        Console.WriteLine("2 + 3 = " + add(2, 3));
        Console.WriteLine("2 * 3 = " + mul(2, 3));
        Console.WriteLine("2 - 3 = " + sub(2, 3));
    }
}

```

§9.2.4 Yerleşik Temsilciler

Her seferinde yeni bir **delegate** tanımlamak yerine, .NET'in hazır yapılarını kullanabiliriz.

- ◊ **Action<T>**: Geriye değer döndürmeyen (**void**) yöntemler için kullanılır.
- ◊ **Func<T, TResult>**: Geriye mutlaka bir değer döndüren yöntemler için kullanılır.

```

// Parametre olarak int alır, geriye string döner
Func<int, string> notaCevir = (puan) => puan >= 50 ? "Geçti" : "Kaldı";
string sonuc = notaCevir(75); // "Geçti"

```

```

Action line = () => Console.WriteLine("-----"); //Parametresiz
line(); //Action delegate'leri, geriye değer döndürmezler!

```

```

Func<double, double> cube = x => x * x * x; //Tek Parametrelili
Console.WriteLine(cube(3));
//Func delegate'leri, geriye değer döndürürler!
/* Yukarıdaki kodun açılımı:
double cube(double x) {
    return x * x * x;
}
*/

```

```

Func<int, double, double> kat = (x,y) => x * y; //İki Parametrelili
Console.WriteLine(kat(3,6.0));

```

```

/* Yukarıdaki kodun açılımı
double kat(int x, double y) {
    return x * y;
}
*/

//Veri tipi içeren parametrelili
Func<int, string, bool> MetinKarakterSayısıVerilendenBüyükMü = (int x, string s)
=> s.Length > x;
Console.WriteLine(MetinKarakterSayısıVerilendenBüyükMü(3, "Merhaba"));

//Parametreler Göz Ardı Edildi
Func<int, int, int> constant = (_, _) => 61;
Console.WriteLine(constant(3, 5)); // Output: 61

```

§9.2.5 Lambda İfadelerinin Gücü

Lambda ifadelerinin asıl parladığı yer LINQ sorgularıdır. Bu konuyu ileride detaylı göreceğiz ama kısa bir örnek verilebilir. Bir diziyi filtrelemek hiç bu kadar kolay olmamıştı:

```

int[] sayılar = { 1, 5, 8, 12, 15, 20 };
var çiftSayılar = sayılar.Where(s => s % 2 == 0); // LINQ ve lambda kullanımı: "s öyle ki s'in
→ 2'ye bölümünden kalan 0 olsun"
foreach (var sayı in çiftSayılar)
    Console.WriteLine($"{sayı} "); // 8 12 20

```

Bir lambda ifadesi, tanımlandığı yöntemin içindeki yerel bir değişkene erişebilir mi? Eğer erişirse ve o değişkenin değeri metodun ilerleyen kısımlarında değişirse ne olur? Evet, buna **kapatma** (**closure**) denir. lambda, dışarıdaki değişkeni "yakalar" (**capture**). Ancak dikkat edilmelidir ki; lambda değişkenin o anki değerini değil, referansını yakalar. Bu yüzden dışarıdaki değişken değişirse lambdanın sonucu da değişecektir.

§9.2.6 Anonim Değişkenlerle Lambda İfadeleri

Bir lambda ifadesini tanımlarken var kelimesi ile temsilci tipi otomatik olarak çıkarılır. Aslında genel olarak bir ifadenin sonucunda ortaya çıkan veri tipinin çıkarımı otomatik olması isteniyorsa anonim değişkenler kullanılır.

```

var IncrementBy = (int source, int increment = 1) => source + increment;
Console.WriteLine(IncrementBy(5)); // 6
Console.WriteLine(IncrementBy(5, 2)); // 7

```

Ayrıca, açıkça yazılmış bir parametre listesindeki son parametre olarak, parametre dizileri veya koleksiyonları olan lambda ifadelerini de tanımlanabilir;

```

var sum = (params IEnumerable<int> values) =>
{
    int sum = 0;
    foreach (var value in values)
        sum += value;
    return sum;
};
var empty = sum();
Console.WriteLine(empty); // 0
var sequence = new[] { 1, 2, 3, 4, 5 };
var total = sum(sequence);
Console.WriteLine(total); // 15

```

9.3 Olaylar

Nesneler, olaylardan (**event**) etkilenirler ve olaylar karşısında tepki verir ya da davranış (**behavior**) gösterirler. Olayı yayan (**publish**) nesneye diğer tüm nesneler tepki vermezler. Bu nedenle tepki verecek nesneler, davranış göstereceği olaya önceden abone (**subscribe**) olurlar. Abone olunan olay gerçekleştiği zaman tepki (**handler**) verirler.

§9.3.1 Olay Tanımlama ve Olay Tetiklendiğinde Tepki Verme

```
//ANA PROGRAM
// 1. Temsilci (Delegate) Tanımı. Olaya abone olmak için kullanılır!
public delegate void OlayOluncaAboneOlunacakYöntem(Kişi s);
internal class Program
{
    static void Main(string[] args)
    {
        Kişi kişi = new Kişi("İlhan", 50);
        Dinleyici dinleyici = new Dinleyici();
        dinleyici.AboneOl(kişi); // Abone olma işlemi
        // Değişiklikler olayı tetikleyecek
        kişi.Yaş = 51;
        kişi.Yaş = 52;
    }
}

//SINIFLAR
public class Kişi
{
    public event OlayOluncaAboneOlunacakYöntem? YaşDeğiştirdi; //Başlangıçta null olmalı!
    /*
    OlayOluncaAboneOlunacakYöntem? olarak nullable tanımlamak istemiyorsak aşağıdaki kodları
    ↪ yazabiliriz:
    1. Boş bir lambda ile başlatıldığı için asla null olmaz:
    public event OlayOluncaAboneOlunacakYöntem YaşDeğiştirdi = delegate { };
    2. Aynı şeyin modern C3.0+ yazımıyla:
    public event OlayOluncaAboneOlunacakYöntem YaşDeğiştirdi = _ => { };
    */
    private int _yaş;
    public Kişi(string pİsim, int pYaş)
    {
        this.İsim = pİsim;
        /*
        DİKKAT: Burada doğrudan '_yaş' mahrem alanına atama yapıyoruz.
        Eğer 'this.Yaş = pYaş' denip Özellik değiştirilseydi, henüz kimse abone (+=)
        olmadan olay tetiklenmeye çalışılır ve hata alabilirsiniz.
        */
        this._yaş = pYaş;
    }
    public string İsim {get; set;}
    public int Yaş
    {
        get => _yaş;
        set
        {
            if (_yaş != value)
            {
                _yaş = value;
                YaşDeğiştirdi?.Invoke(this); // 'Null-conditional' operatörü (?.) kullanarak daha
                ↪ güvenli tetikleme:
            }
        }
    }
}

public class Dinleyici
{
    public void AboneOl(Kişi k)
    {
        k.YaşDeğiştirdi += Kişinin_yaşıDeğişinceNeYapılacak; // += ile olaya kayıt olunur
    }
}
```

```

private void Kişinin_yaşıDeğişinceNeYapılacak(Kişi kişi)
{
    Console.WriteLine($"{kişi.İsim} adlı kişinin yeni Yaşı: {kişi.Yaş}");
}

```

§9.3.2 Yerleşik Olaylar

C# dilinde yerleşik olarak olaylar parametre alacak şekilde aşağıdaki gibi tanımlanır;

```
public event EventHandler<EventArgs> EventName;
```

<EventArgs> için jenerik başlığını inceleyebilirsiniz. Olaylara tepki verecek yöntemler aşağıdaki gibi tanımlanır;

```

public void HandlerName(object sender, EventArgs args)
{
    /* Handler logic */
}

```

Olaya abonelik de aşağıdaki gibi yapılır;

```
EventName += HandlerName; // Olaya Abone Olma
```

Eğer bir **Windows Form** projesinde yazıyorsanız, Form üzerinde sağ tıklayıp özelliklerine giderseniz form ile ilgili olay listesini görebilirsiniz. Herhangi birine çift tıklayarak ona ilişkin tepki yöntemi otomatik olarak kodlanacak şekilde oluşacaktır. Aşağıda örneği verilmiştir;

```

//Form1.cs
namespace WinFormsApp1
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
        private void Form1_Load(object sender, EventArgs e)
        {

        }
        private void Form1_Click(object sender, EventArgs e)
        {

        }
        private void Form1_DoubleClick(object sender, EventArgs e)
        {

        }
    }
}

```

Yukarıda verilen **Kişi** sınıfı örneğini yerleşik <EventArgs> ile aşağıdaki gibi yeniden yazabiliriz;

```

//ANA PROGRAM:
internal class Program
{
    static void Main(string[] args)
    {
        Kişi kişi = new Kişi("İlhan", 50);
        Dinleyici dinleyici = new Dinleyici();

        dinleyici.AboneOl(kişi);

        kişi.Yaş = 51; // Çıktı: İlhan adlı kişinin yaşı değişti! 50 -> 51
        kişi.Yaş = 52; // Çıktı: İlhan adlı kişinin yaşı değişti! 51 -> 52
    }
}

```

```
// 1. Standart Temsilci Tanımı
// object sender: Olayı tetikleyen nesne
// YaşDeğişmeOlayıArgs args: Olay ile ilgili veriler
public delegate void YaşDeğişinceAboneOlunacakYöntem(object sender, YaşDeğişmeOlayıArgs args);

//SINIFLAR:
// 2. Veri Taşıyıcı Sınıf (EventArgs)
public class YaşDeğişmeOlayıArgs : EventArgs
{
    public int MevcutYaş { get; }
    public int YeniYaş { get; }

    public YaşDeğişmeOlayıArgs(int mevcutYaş, int yeniYaş)
    {
        MevcutYaş = mevcutYaş;
        YeniYaş = yeniYaş;
    }
}

// 3. Yayıncı Sınıf (Publisher)
public class Kişi
{
    public event YaşDeğişinceAboneOlunacakYöntem? YaşDeğişti;

    private int _yaş;
    public string İsim { get; set; }

    public Kişi(string pİsim, int pYaş)
    {
        İsim = pİsim;
        _yaş = pYaş; // Doğrudan field'a atama (Olay tetiklenmez)
    }

    public int Yaş
    {
        get => _yaş;
        set
        {
            if (_yaş != value)
            {
                int eskiYaş = _yaş; // Önce eski değeri saklıyoruz
                _yaş = value;       // Sonra yeni değeri atıyoruz

                // Olayı argümanlarla birlikte tetikliyoruz
                YaşDeğişti?.Invoke(this, new YaşDeğişmeOlayıArgs(eskiYaş, value));
            }
        }
    }
}

// 4. Abone Sınıf (Subscriber)
public class Dinleyici
{
    public void AboneOl(Kişi k)
    {
        k.YaşDeğişti += KişininYaşıDeğişinceYapılacak;
    }

    private void KişininYaşıDeğişinceYapılacak(object sender, YaşDeğişmeOlayıArgs e)
    {
        // sender nesnesini Kişi tipine güvenli bir şekilde dönüştürebiliriz
        if (sender is Kişi k)
        {

```

```

        Console.WriteLine($"{k.İsim} adlı kişinin yaşı değişti! Eski: {e.MevcutYaş}, Yeni:
        ↪ {e.YeniYaş}");
    }
}

```

§9.3.3 Olaylara Tepki Vermek İçin Lambda İfadeleri

Lambda ifadeleri olaylara tepki vermek için kullanılabilir. Bu durumda tepki gösterilecek olaya abone (**subscribe**) olduktan sonra abonelikten ayrılma söz konusu olmaz. Ana programı inceleyiniz!

internal class Program

```

{
    static void Main(string[] args)
    {
        Kişi kişi = new Kişi();
        Dinleyici dinleyici = new Dinleyici();
        dinleyici.AboneOl(kişi);
        kişi.Yaş = 20;
        kişi.Yaş = 30;
        Console.WriteLine("----- ");
        YaşDeğişinceAboneOlunacakYöntem tepki = (sender, args)
        => Console.WriteLine("Kişinin Yaşı Değişemez!!");
        kişi.yaşDeğişti += tepki; //kişi.yaşDeğişti -= tepki; talimatı yazılarak abonelikten
        ↪ çıkılabilir.
        kişi.yaşDeğişti += (sender, args) => Console.WriteLine("Kişinin Yaşı Savcılık Emriyle
        ↪ Değişebilir!"); // Bu tepki için aynı şey yapılamaz!
        kişi.Yaş = 40;
    }
}
public class YaşDeğişmeOlayıArgs : EventArgs
{
    public YaşDeğişmeOlayıArgs(int mevcutyaş, int yeniyaş)
    {
        this.MevcutYaş = mevcutyaş;
        this.YeniYaş = yeniyaş;
    }
    public int MevcutYaş { get; private set; }
    public int YeniYaş { get; private set; }
}
public delegate void YaşDeğişinceAboneOlunacakYöntem(object sender, YaşDeğişmeOlayıArgs args);
public class Kişi
{
    public Kişi()
    {
        yaş = 0;
        İsim = "İsimsiz";
        yaşDeğişti = null;
    }
    public event YaşDeğişinceAboneOlunacakYöntem yaşDeğişti;
    public string İsim { get; set; }
    private int yaş;
    public int Yaş
    {
        get { return yaş; }
        set
        {
            if (yaş != value)
            {
                if (yaşDeğişti != null)
                {
                    YaşDeğişmeOlayıArgs args = new YaşDeğişmeOlayıArgs(yaş, value);
                    yaşDeğişti(this, args);
                }
            }
        }
    }
}

```



```

        }
        yaş = value;
    }
}
}
}
public class Dinleyici
{
    /* "Dinleyici" sınıfından imal edilen nesneler,
    * "Kişi" sınıfından imal edilen nesnelere abone olabilir. */
    public void AboneOl(Kişi k)
    {
        k.yaşDeğiştirdi += this.kişininYaşıDeğişinceYap;
    }
    public void kişininYaşıDeğişinceYap(object sender, YaşDeğişmeOlayıArgs args)
    {
        Console.WriteLine($"{args.MevcuyYaş} olan Kişinin Yaşı " +
            $"{args.YeniYaş} olarak Değişti!");
    }
}
}

```

Olaylara += ile yapılan aboneliklerin, nesneyle iş bitince -= ile sonlandırılmaması durumunda oluşabilecek bellek sızıntılarına yol açar!

§9.3.4 Modern ve Standart Olay Kullanımı

C# dilinde her olay için yeni bir `delegate` tanımlamanıza gerek yoktur.

Bunun için .NET içindeki hazır `EventHandler<TEventArgs>` yapısını kullanarak kodunuzu daha da sadeleştirir bilirsiniz:

```

internal class Program
{
    static void Main()
    {
        Kişi ilhan = new Kişi("İlhan", 50);
        Dinleyici takipçi = new Dinleyici();
        takipçi.AboneOl(ilhan);
        // Değişiklikler otomatik olarak bildirilecek
        ilhan.Yaş = 55;
        ilhan.Yaş = 60;
    }
}
// 1. Olay Veri Sınıfı: EventArgs'tan türetilmelidir.
public class YaşDeğişmeOlayıArgs : EventArgs
{
    public int EskiYaş { get; }
    public int YeniYaş { get; }
    public YaşDeğişmeOlayıArgs(int eskiYaş, int yeniYaş)
    {
        EskiYaş = eskiYaş;
        YeniYaş = yeniYaş;
    }
}
// 2. Yayıncı Sınıf (Publisher)
public class Kişi
{
    // Standart EventHandler kullanımı: Ayrı bir delegate tanımına gerek duymaz.
    public event EventHandler<YaşDeğişmeOlayıArgs>? YaşDeğiştirdi;
    private int _yaş;
    public string İsim { get; set; }
    public Kişi(string isim, int yaş)
    {

```

```

        İsim = isim;
        _yaş = yaş;
    }
    public int Yaş
    {
        get => _yaş;
        set
        {
            if (_yaş != value)
            {
                int eski = _yaş;
                _yaş = value;
                // Olayı tetiklerken 'this' gönderen nesneyi,
                // yeni YaşDeğişmeOlayıArgs ise veriyi temsil eder.
                YaşDeğiştirdi?.Invoke(this, new YaşDeğişmeOlayıArgs(eski, value));
            }
        }
    }
}
// 3. Abone Sınıf (Subscriber)
public class Dinleyici
{
    public void AboneOl(Kişi k)
    {
        k.YaşDeğiştirdi += KişininYaşıDeğiştirdiğinde; // Olay aboneliği
    }
    // Standart imza: (object sender, TEventArgs e)
    private void KişininYaşıDeğiştirdiğinde(object? sender, YaşDeğişmeOlayıArgs e)
    {
        if (sender is Kişi k)
        {
            Console.WriteLine($"[BİLGİ] {k.İsim} adlı kişinin yaşı güncellendi.");
            Console.WriteLine($" => Eski Yaş: {e.EskiYaş}");
            Console.WriteLine($" => Yeni Yaş: {e.YeniYaş}\n");
        }
    }
}
}

```

9.4 Düşün ve Çöz

- ◇ **Düşün:** Bir temsilciye (**delegate**) üç farklı metot bağladınız. Eğer bu metotlardan ikincisi çalışma anında bir istisna (**exception**) fırlatırsa, üçüncü metot çalışmaya devam eder mi? Neden?
- ◇ **Düşün:** Neden olayları (**event**) doğrudan bir temsilci değişkeni (**delegate variable**) gibi **public** olarak tanımlayıp dışarıdan erişime açmıyoruz da **event** anahtar kelimesini kullanıyoruz? (Kısıtlama farkları nelerdir?)
- ◇ **Düşün:** **Action** ve **Func** arasındaki temel fark nedir? Hangisi geriye değer döndüren metotlar için kullanılır?
- ◇ **Düşün:** Temsilci (**delegate**) nedir ve C#’ta fonksiyon işaretçisi (**function pointer**) ihtiyacını nasıl karşılar? Nesne yönelimli bir dilde fonksiyon işaretçisine sahip olmanın faydası ne olabilir?
- ◇ **Düşün:** Çok noktaya yayın temsilcisi (**multicast delegate**) nasıl çalışır? Birden fazla yöntemi zincirlemenin hangi gerçek dünya senaryolarında faydalı olduğunu açıklayınız. Zincirden bir yöntem çıkarmak nasıl mümkündür?
- ◇ **Düşün:** Lambda ifadesi ile anonim yöntem (**anonymous method**) arasındaki söz dizimi farkı nedir? Lambda ifadelerinin okunabilirlik açısından üstünlüğünü bir örnek üzerinde gösterin.
- ◇ **Düşün:** **Func<T>**, **Action<T>** ve **Predicate<T>** yerleşik temsilcileri arasındaki farklar nelerdir? Hangisinin dönüş değeri vardır, hangisi yoktur? Gerçek bir senaryo üzerinden her birinin kullanım durumunu belirtiniz.
- ◇ **Düşün:** Olay ile temsilci arasındaki temel fark nedir? ‘event’ anahtar kelimesi olmadan bir temsilci alanı kullanmanın güvenlik riski nedir? Neden doğrudan temsilci yerine olay tercih edilir?
- ◇ **Düşün:** Olayların abonelik (**subscribe**) ve abonelik iptali (**unsubscribe**) yönetimi neden önemlidir? Olay aboneliğinin iptal edilmemesi bellek sızıntısına nasıl yol açar? Bu sorunu önlemenin yolları nelerdir?
- ◇ **Düşün:** **EventArgs** ve özel olay argümanı sınıflarının (**custom EventArgs**) rolü nedir?

Standart `EventHandler<TEventArgs>` imzasını kullanmanın avantajları nelerdir?

- ◇ **Düşün:** Lambda ifadelerinde dışarıdaki değişkenlerin yakalanması (**closure**) nasıl çalışır? Bir döngü içinde lambda ifadesi kullanıldığında ne tür tuzaklarla karşılaşılabilir? Bu tuzaktan nasıl kaçınılır?
- ◇ **Çöz:** Sıralama stratejisi dinamik olarak değiştirilebilen bir ürün listesi tasarlayınız: kullanıcı 'ada göre', 'fiyata göre' veya 'stoka göre' sıralama seçeneği sunulsun. Her sıralama kriterini bir `Func<Ürün, object>` temsilcisiyle ifade edin.
- ◇ **Çöz:** Fare tıklaması gibi davranan bir `Button` sınıfı oluşturunuz. `Clicked` adında bir olay tanımlayın; birden fazla dinleyici abone olabilsin ve olay tetiklendiğinde hepsi haberdar edilsin.
- ◇ **Çöz:** Bir sayı listesi üzerinde filtreleme ve dönüştürme işlemlerini lambda ifadesiyle yapan program yazınız: çift sayıları filtreleyin, karesini alın ve toplamı hesaplayın.
Önce `foreach` ile, sonra `Func/Predicate` ile yazın; okunabilirlik farkını karşılaştırın.
- ◇ **Çöz:** `EventArgs`'tan türetilen `SicaklikDegistiEventArgs` sınıfını oluşturunuz. Bir Termometre sınıfı tanımlayın; sıcaklık belirli bir eşiği aştığında olay fırlatsın ve bir alarm sistemi bu olaya abone olsun.
- ◇ **Çöz:** `Action<string>` temsilcisini kullanarak birden fazla günlük (**log**) kaydedici (konsol, dosya, veritabanı simülasyonu) arasında seçim yapılabilen esnek bir günlükleme sistemi tasarlayınız.